Software Requirements and Design Document

For

Group <RuneScape Web App>

Version 2.0

Authors:
Agustin Camus
Colton Bregoff
Miles Burkart
Ricardo Furst
William Kilduff

1. Overview (5 points)

RuneReport is a web application for Old School RuneScape players who trade items on the Grand Exchange. Instead of telling players what to buy right now, it lets them test trading strategies against real historical price data. A user finds an item, enters conditions for buying and selling on that item's page, and generates a report showing how the strategy would have performed after accounting for in-game constraints such as the 2% Grand Exchange sales tax. Reports can be saved and revisited to learn from them. The application is a static, client-side website with no backend and no user accounts. Market data comes from the OSRS Wiki Real-time Prices API.

2. Functional Requirements (10 points)

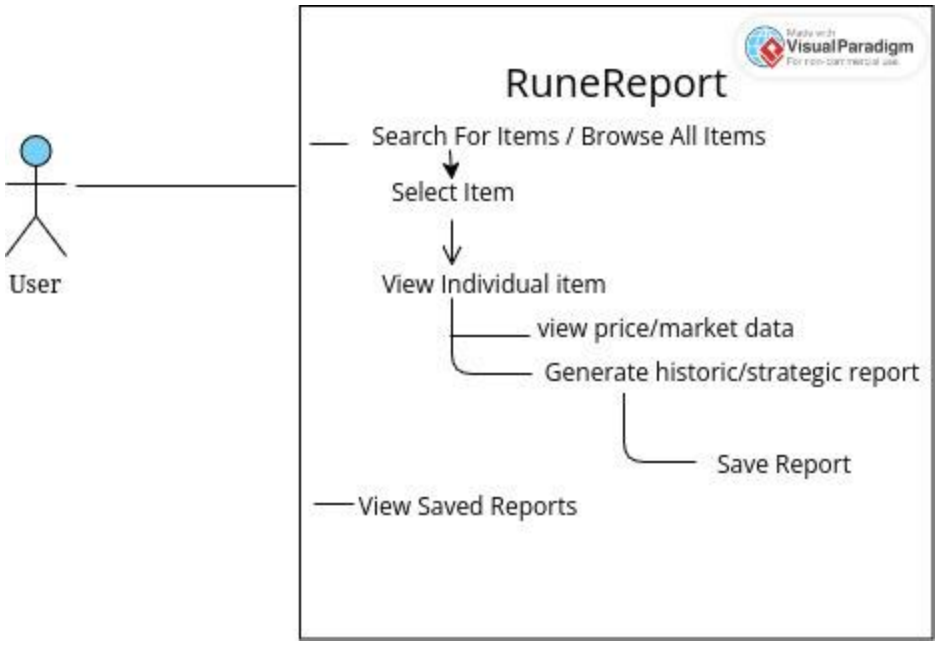
1. Strategy tester algorithm that takes user-inputted conditions from a dedicated item page and compares it against historical data to determine how well it would have done - **High Priority**
2. Item search feature on homepage displays live search results, pulled from the API, with clickable items that bring the user to a dedicated item page - **Medium Priority**
3. Most Traded Today panel on homepage pulls 5 items from the OSRS API with highest trade volume, refreshing periodically (perhaps every 5 or 10 minutes) - **Medium Priority**
4. Search feature on browse page must function like the homepage search bar, displaying live results from the API with clickable items that bring the user to a dedicated item page - **Medium Priority**
5. Sort feature on browse page that sorts full list of items based on conditions such as Price, Daily Volume, and High Alchemy Price - **Low Priority**
6. Items on browse page display all items with descriptions from the OSRS API, dividing them among pages as needed - **High Priority**
7. Numbered navigation bar at the bottom of the browse page brings the user to the correct page based on the number/arrow they click - **Low Priority**
8. Items on browse page need to be frequently updated according to data pulled from the API - **Medium Priority**
9. Dedicated item pages frequently update their display data (e.g. Buy Price, Sell Price, Margin, Daily Volume, etc.) based on values pulled from the API - **Medium Priority**

10. Condition fields under the Report Settings panel on dedicated item pages feed inputs to the strategy tester algorithm - **High Priority**
11. Generate Report button runs the strategy tester algorithm, generating a report of how well the user-inputted strategy would have worked based on historical market data - **High Priority**
12. Save report button on the item report page saves a generated report to the Saved Reports page; this data should persist between sessions - **Medium Priority**
13. Saved Reports page should display the user's saved reports as clickable elements in order of how recently they were created (with newest reports at the top), and divide the reports among pages as needed - **Medium Priority**
14. Numbered navigation bar at the bottom of the saved reports page brings user to the correct page based on the number they click - **Low Priority**

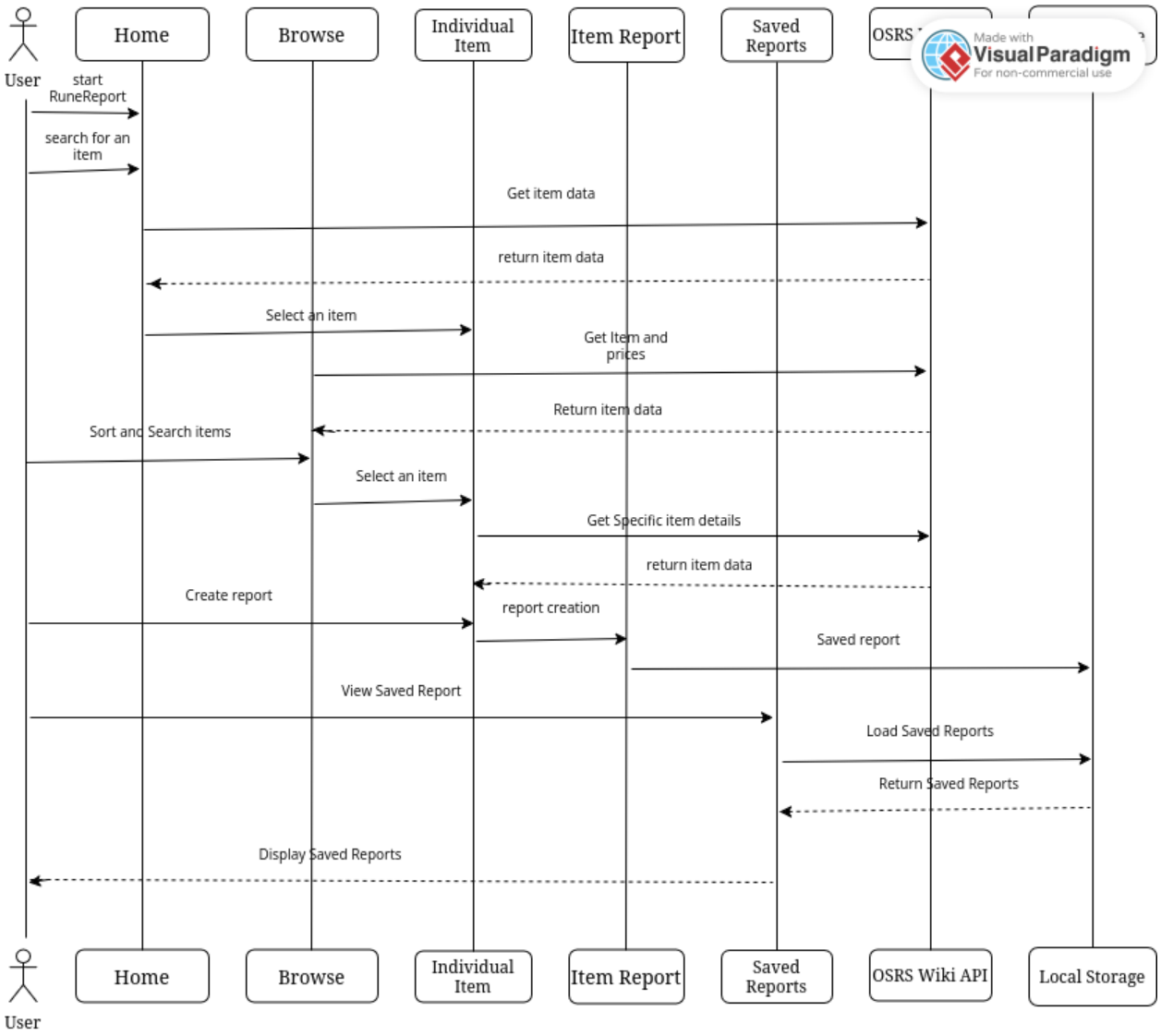
3. Non-functional Requirements (10 points)

1. Performance: Quick API response time for populous item page, on typical broadband connection, page shall finish loading API data within 3 seconds.
2. Data Integrity: The system shall display prices exactly as returned by the API, without alteration.
3. API Compliance: the system shall follow the OSRS Wiki Real-time Prices API usage guidelines, including limiting how often it requests data.
4. Accuracy: profit calculation needs to account for in-game variables, like GE tax
5. Privacy: the system shall not collect personal information.
6. Reliability: the system shall handle API failures without crashing.

4. Use Case Diagram (10 points)



5. Class Diagram and/or Sequence Diagrams (15 points)



6. Operating Environment (5 points)

RuneReport is a static, client-side website built with HTML, CSS, and JavaScript. It runs in a web browser on Windows, macOS, or Linux, and needs an internet connection to load data from the OSRS Wiki Real-time Prices API. It is designed for current versions of Chrome, Firefox, Edge, and/or Safari on desktop (with intent to work on mobile browsers too). We will find a platform to host the website further on, such as GitHub Pages.

7. Assumptions and Dependencies (5 points)

Assumptions:

- The OSRS Wiki Real-time prices API will remain available and keep its current format.
- The API provides enough historical data to produce meaningful reports.
- Users' browsers support local storage, which is used to keep saved reports between sessions.

Dependencies:

- The OSRS Wiki Real-time prices API for all item and price data.
- The OsFont library and the RuneScape Chat Font for the game-style text.
- GitHub for source control, issue tracking, and hosting.